

Founding Engineer: Take-Home Exercise

RFQ Data Ingestion

Thanks for taking the time. This exercise is a small, self-contained slice of a problem we work on every day. We care far more about how you think than about how many features you cram in. The judgement calls, the tradeoffs, and the way you handle messy reality are what we are looking at.

Time budget: roughly 4 hours. Please don't go far beyond that. If you run out of time, a clean and well-explained core beats a sprawling, half-working pile of features. Tell us what you would do next instead of doing it.

Background

We work in **electronic component distribution**. Manufacturers and contract assemblers build circuit boards, and to do that they buy parts: microcontrollers, op-amps, resistors, capacitors, connectors, and so on. Each part has a **manufacturer part number** (MPN) like LM358N or STM32F407VGT6, made by a manufacturer such as Texas Instruments or STMicroelectronics. The same part often comes in several package or grade variants (LM358N vs LM358D), and buyers refer to manufacturers by shorthand ("TI", "ON Semi") as readily as by full name.

When a buyer needs parts, they send a **Request for Quote (RFQ)**: an email asking a distributor to price a list of parts, each with a quantity and sometimes a target price, manufacturer, or required date. RFQs arrive as free-form text, tables, spreadsheets, PDFs, even scanned images, and quantities range from a handful of prototypes to hundreds of thousands of pieces on a reel.

Today a person reads each RFQ and re-types the parts, quantities, and customer details into an ERP system by hand. We automate that: an email comes in, and a structured requirement comes out the other side, ready to be created in the ERP.

Your job is to build the heart of that pipeline.

The core task (required)

Build a small service that turns a raw RFQ email into structured data.

1. **Ingest an email.** We give you a folder of raw .eml files in `samples/`. Your service reads one and works with its contents. Some RFQs carry the parts list in an attached **CSV or PDF** rather than in the email body, so pull the data from wherever it lives.
2. **Decide whether it is an RFQ.** Not every email is. Some are order confirmations, newsletters, or support questions. Those must be recognised and skipped.
3. **Extract the structured requirement** from the ones that are RFQs, using a large language model of your choice.
4. **Expose it over HTTP.** Provide an endpoint, `POST /ingest`, that accepts the contents of one

email and returns the structured result described below.

5. **Show it in a dashboard.** A minimal web UI that lists the RFQs you have ingested and shows the extracted fields in a readable way. Think "an operator glances at this to see what came in this morning." Plain and clear beats fancy and broken.

Several samples contain genuine ambiguity with no single correct answer: quantity ranges, "or equivalent" alternates, grouped or per-board quantities, and details split between the email body and an attachment. We are far more interested in the call you make and how you justify it than in a perfect answer. Use the `warnings` field and your README to surface those judgement calls.

Treat the contents of every email as untrusted input from the outside world: it is fed straight to a model, and not all of it is well behaved.

Wrap it up with a short **README**: how to run it, the key decisions you made and why, and what you would do with more time.

The output contract

Every email your service ingests must produce JSON in this shape. This is the contract the rest of our system depends on: downstream code maps it straight into the ERP, so the structure matters as much as the content.

For an email that is an RFQ:

```
{
  "isRfq": true,
  "confidence": 0.0,           // 0 to 1: how sure you are about this extraction
  "customer": {
    "name": "string | null",
    "email": "string | null",
    "phone": "string | null",
    "company": "string | null"
  },
  "request": {
    "dueDate": "YYYY-MM-DD | null",
    "priority": "low | medium | high | urgent",
    "specialInstructions": "string | null"
  },
  "lineItems": [
    {
      "partNumber": "string",
      "manufacturer": "string | null",
      "description": "string | null",
      "quantity": 0,           // integer; 0 if not stated
      "targetPrice": 0.0,      // number | null
      "notes": "string | null"
    }
  ],
  "warnings": ["string"]      // anything ambiguous worth flagging to a human
}
```

For an email that is NOT an RFQ:

```
{
  "isRfq": false,
  "confidence": 0.0,
  "reason": "string"           // one line on why it is not an RFQ
}
```

Getting a language model to reliably produce data in this shape, every time and on messy input, is a real part of what we are interested in. How you achieve that is for you to work out.

Optional tasks

Option A: A verification pass

Add a check that runs after extraction and flags any line item whose part number or quantity does not actually appear in the source email or attachment. The point is to catch a model that invents or mis-transcribes a part before it reaches the ERP. Surface a flagged item for a human; do not silently drop it.

Option B: Read an RFQ from an image

One sample carries its parts list as an image attachment, a scanned table with no text layer, the way a customer photo or scan arrives. Extract the parts from it. This needs a vision-capable model, and it is the most involved option, so treat it as a stretch beyond the core budget.

Option C: Make it your own

Build something that shows us how you think. This is the open one. A few sparks, not a checklist:

- **Agentic tool-use:** let the model call tools, e.g. `lookup_part` against `samples/parts-catalog.csv`, or a calculator for per-board quantities ("2 per board, 500 boards").
- **Self-critique:** a second pass that reviews the first extraction for missed or invented line items and revises it.
- **Normalization:** resolve messy part numbers and manufacturer aliases (TI, ON Semi) to canonical entries in `samples/parts-catalog.csv`.
- **Provenance:** have each line item point back to the exact text it came from.

Pick one, combine a few, or do something we did not think of. Tell us why it was worth building.

What we provide

```
samples/
  rfq-*.eml      raw RFQ emails; some carry their parts list in an
                  attached CSV, PDF, or image rather than the email body
  not-an-rfq-*.eml emails that should be skipped
  labels.json    hand-checked answers for five of the emails
  parts-catalog.csv a small canonical parts list (for Option C)
```

You do not need any of our systems, accounts, or credentials. Everything you need is in this bundle.

Constraints and non-goals

- **Use any language, framework, and LLM provider you like.** Pick what you are fastest and happiest in. We want to see what you reach for.
 - **Bring your own LLM key.** Use your own API key for whichever provider you choose, and **never commit it**. Read it from an environment variable. If you genuinely cannot access any model, leave a clear note and stub the call so the rest of the service still runs.
 - **No database required.** In-memory or a local file is completely fine.
 - **Explicitly out of scope.** Please do not spend time here: authentication, deployment, real ERP integration, Excel (.xlsx) attachments, and multi-user concerns.
-

What to submit

A git repository (a link or a zip) containing:

1. Your code.
2. A `README.md` covering: how to run it, the decisions and tradeoffs you made, anything you would improve with more time, and roughly how long you spent.

Please send it over **a few hours before your interview**, so we have time to read through it before we talk.

On the call

We will start with a short **demo, 15 to 20 minutes**: run us through what you built, then walk us through the code and the decisions behind it. After that we move into a technical discussion, digging into the tradeoffs, what you would change, and where you would take it next.

How we evaluate

This is a founding-engineer hire, so we look across the whole picture:

- **Does it work?** Correct, sensible extractions on the samples, including the messy one and the non-RFQ.
- **Judgement on messy input.** How you handle ambiguity, missing fields, and things the model gets wrong.
- **Code we would want to live with.** Clear structure, readable, honest error handling.
- **How you work with a language model.** Reliability and robustness to untrusted input, not just a happy-path prompt.
- **Communication.** A README that tells us why, not just what.

There are no trick questions. Build the thing you would be comfortable shipping a first version of, and tell us where you would take it next. Good luck. We are looking forward to seeing how you think.